

Backup, Disaster Recovery & Deployment Strategies

Phase 3 · Advanced · Estimated time: 1.5 hours

1. Concept From First Principles

This chapter covers two related but distinct practical concerns that every production system needs an explicit answer for: how do you recover data after something goes catastrophically wrong (backup and disaster recovery), and how do you safely ship new code without that shipping process itself becoming a source of outages (deployment strategies)? Both are less glamorous than consensus algorithms or event sourcing, but interviewers who've actually operated production systems weight them heavily, because they're where real incidents actually happen.

Backups: RPO and RTO

A **backup** is a point-in-time copy of your data, kept separately from the live system, so that data loss, corruption, or a catastrophic failure doesn't mean the data is gone forever. Two metrics define what "good enough" backup strategy actually means for a given system, and interviewers expect you to use this exact vocabulary. **RPO (Recovery Point Objective)** is how much data you can afford to lose, measured in time — an RPO of 1 hour means your worst-case data loss, in a disaster, is up to 1 hour's worth of changes since the last backup. **RTO (Recovery Time Objective)** is how long you can afford to be down while recovering — an RTO of 30 minutes means the system must be back up and serving traffic within 30 minutes of a disaster being declared. These two numbers directly drive backup frequency and disaster-recovery architecture: a payments ledger might demand an RPO of seconds (achieved via continuous replication, not periodic backups, directly connecting to Phase 2 Chapter 5's synchronous replication) and an RTO of minutes, while an internal analytics database might tolerate an RPO of a full day and an RTO of several hours.

Backend engineer analogy

This is the difference between saving a document you're actively editing every few seconds (near-zero RPO, but you need auto-save infrastructure) versus a weekly manual backup of your whole project folder (RPO of up to a week — you'd lose up to a week of work in the worst case, but it's much simpler and cheaper to maintain). Neither is universally 'correct' — the right RPO/RTO depends entirely on how expensive losing that data or that uptime actually is for the specific system.

It's worth stating a rule that experienced engineers learn the hard way, often after a real incident: **a backup you haven't tested restoring is not a real backup**. It's extremely common for organizations to run backups faithfully for years and only discover, during an actual emergency, that the backup was corrupted, incomplete, or that the restore process itself was never actually documented or rehearsed. Disaster recovery planning genuinely requires periodic **restore drills** — actually practicing the recovery process end to end, on a schedule, before a real disaster forces you to discover the gaps live.

Deployment strategies: shipping code without shipping outages

Rolling deployment updates instances of a service one at a time (or in small batches), keeping the overall service available throughout — simple and resource-efficient, but a bad deploy still affects some fraction of live traffic before it's caught and rolled back, and mixing old and new code versions serving traffic simultaneously during the rollout requires the two versions to be compatible.

Blue-green deployment maintains two complete, identical production environments ("blue" currently live, "green" idle), deploys the new version fully to the idle environment, tests it thoroughly while it receives zero live traffic, then switches all traffic to it in one atomic cutover — giving a clean, instant rollback (just switch traffic back to blue) if anything goes wrong, at the cost of running two full production environments simultaneously, which roughly doubles infrastructure cost during the deployment window.

Canary deployment releases the new version to a small percentage of live traffic first (1%, then 5%, then 25%, then 100%, monitoring metrics at each stage), catching problems while they're only affecting a small fraction of real users, before a full rollout. This directly reuses this Phase's Chapter 13 observability chapter as a hard prerequisite — a canary deployment without solid metrics and alerting to actually detect a problem in that small percentage of traffic is just a slower, false-confidence version of a regular rollout, since nobody's actually watching for the signal that would justify stopping the rollout.

The connective insight tying deployment strategies to disaster recovery: both are fundamentally about **controlling blast radius** — limiting how much of the system, and for how long, is affected when something goes wrong, whether that something is a bad code deploy or a genuine infrastructure disaster. A canary deployment is blast-radius control applied to code changes; a well-architected multi-region disaster recovery plan (Chapter 11) is blast-radius control applied to infrastructure failures. Recognizing this as the same underlying principle, applied at different layers, is exactly the kind of connective, systems-level thinking this program has been building toward across all three phases.

2. Real-World Example / Case Study

Razorpay's and PhonePe's ledger systems demand an extremely tight RPO — effectively seconds, achieved through synchronous or near-synchronous replication (Phase 2 Chapter 5) rather than periodic backups alone — because even a few minutes of lost financial transaction data represents real, unacceptable money and trust at stake, unlike a system where periodic backups genuinely suffice.

Swiggy's and Zomato's mobile-app-facing backend services commonly use canary deployments for user-facing changes, rolling a new version out to a small percentage of traffic in one city or region first, watching order-success-rate and error-rate metrics (Chapter 13) closely, before expanding to full traffic — catching a bad release while it's only affecting a small, contained group of real orders rather than the entire platform.

A STC- or Tamara-scale fintech, subject to regulatory expectations around business continuity, would typically be required to demonstrate documented, periodically-tested disaster recovery procedures (restore drills, defined RPO/RTO targets, and evidence of having actually rehearsed them) as part of compliance audits — a good reminder that disaster recovery in regulated financial contexts isn't just an engineering best practice, it's often a hard legal and compliance requirement.

3. Diagram

```

Last backup      DISASTER      Recovery complete
|-----RPO----->|<-----RT0----->|
|               |               |
(data up to     (data lost:     (system back up,
this point      everything       serving traffic
is safe)        after last      again)
                backup)

Canary deployment rollout:

v1 (100% traffic) -> deploy v2 to 1% -> watch metrics (Ch 13)
|                                     |
|                                     healthy? --> expand to 5%, 25%, 100%
|                                     unhealthy? --> ROLLBACK, only 1%
|                                     of users were ever affected
d
Blue-Green: [Blue: v1, LIVE] <-switch atomically-> [Green: v2, tested, idle-
>LIVE]
```

Strategy	Blast radius on bad deploy	Cost / Complexity
Rolling deployment	Partial — some instances serve bad version briefly	Low cost, moderate risk if not caught quickly
Blue-green deployment	Full, but instant rollback available	Doubles infrastructure cost during deploy window
Canary deployment	Minimal — only a small traffic percentage initially	Needs strong observability (Ch 13) to be effective
No staged rollout (deploy to 100% at once)	Full, immediate, no early warning	Cheapest, riskiest

Q3: "Compare canary and blue-green deployment. When would you choose each?"

Model approach: Canary limits blast radius by traffic percentage and needs strong observability to work; blue-green gives instant full rollback at double infrastructure cost. Choose canary for gradual, metric-driven confidence building; choose blue-green when you need a guaranteed instant, clean rollback and can afford the duplicate infrastructure.

Q4: "Why does a canary deployment strategy depend on having good observability already in place?"

Model approach: Explain that a canary's entire value comes from detecting a problem in the small percentage of affected traffic before expanding — without solid metrics and alerting (Chapter 13), there's no reliable signal to catch a bad release, making the canary stage a false sense of safety rather than real protection.

6. Practice Exercises

- 1 Define target RPO and RTO for a food delivery app's order database, and justify your numbers with concrete business reasoning.
- 2 Design a canary rollout plan (stages, percentages, metrics to watch, rollback trigger) for a checkout-flow code change.
- 3 Explain why a company might choose blue-green deployment for its payment-processing service specifically, despite the doubled infrastructure cost.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Deployment Strategies Explained: Blue-Green vs. Canary vs. Rolling"	Direct comparison of all three deployment strategies this chapter covers.
English	"Deployment Strategies Explained Blue/Green, Rolling, Canary & Hotfix"	Reinforces the trade-offs with additional real-world framing.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers deployment strategies as part of a full bilingual course.
Hindi	"Complete System Design Roadmap 2026 HLD & LLD" — CodeHelp	Places backup/DR and deployment strategy in context of the broader Hindi HLD roadmap.

8. Key Takeaways

- RPO defines how much data loss is acceptable; RTO defines how much downtime is acceptable, both measured in time.
- Tighter RPO/RTO targets require more expensive, continuous replication rather than periodic backups alone.
- A backup that's never been restore-tested should not be trusted; restore drills are a real, necessary practice.
- Rolling deployments update instances gradually; blue-green switches all traffic atomically with instant rollback.

- Canary deployments limit blast radius by traffic percentage, but depend on strong observability (Chapter 13) to work.
- Deployment strategy and disaster recovery are both fundamentally about controlling blast radius, at different layers.
- Regulated industries (fintech) often require documented, periodically-tested DR procedures as a compliance necessity.